

TASK 1

Q1

设计一个程序 实现功能: 将doro.jpg复制一份到项目根目录下 新图片命名为doro_copy.jpg

这题其实还挺顺利的，唯一的一次报错在while的括号没写好

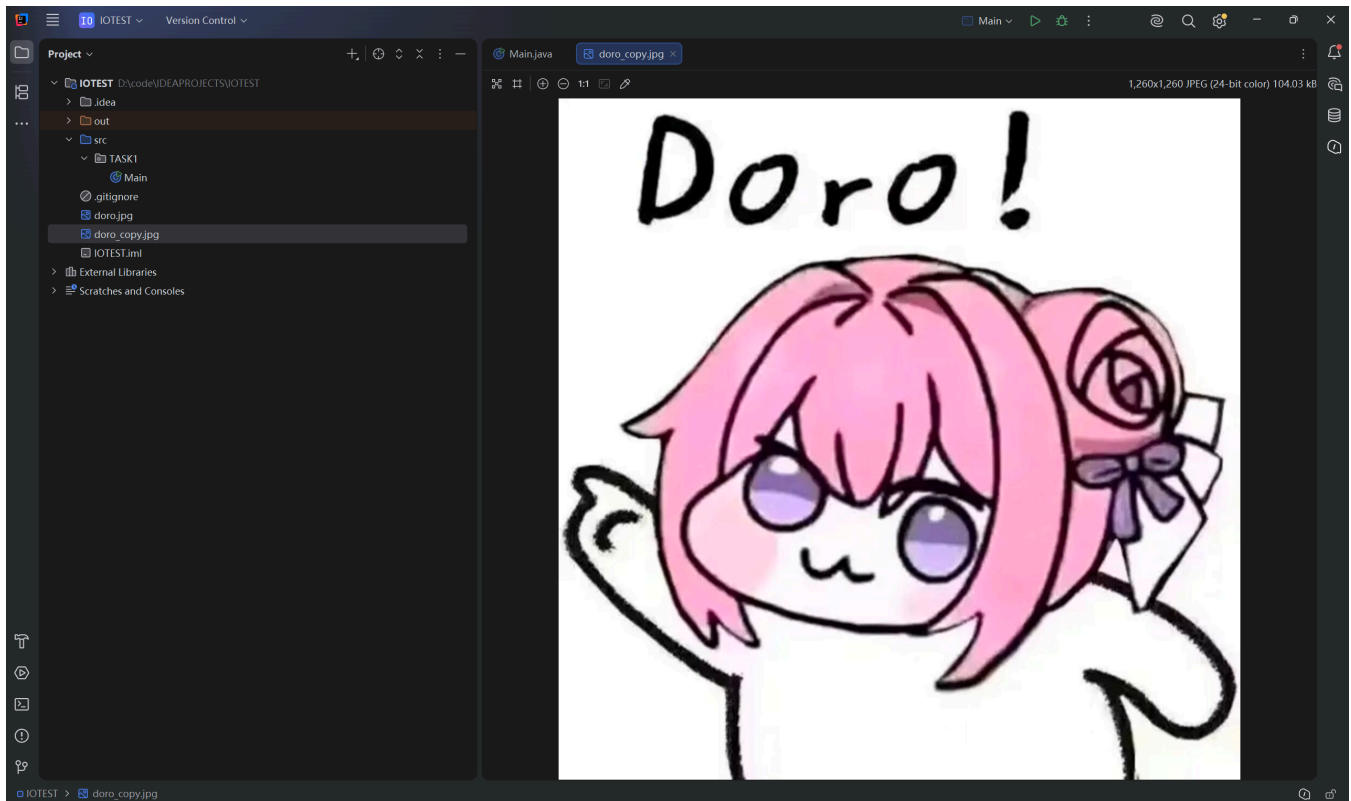
测试代码

```
package TASK1;

import java.io.FileInputStream;
import java.io.FileOutputStream;
import java.io.IOException;

public class Main {
    public static void main(String[] args) throws IOException {
        //使用字节流复制文件
        //由于是在项目根目录，这里就直接写相对路径了
        FileInputStream fis = new FileInputStream("doro.jpg");
        FileOutputStream fos = new FileOutputStream("doro_copy.jpg");
        //先创建一个byte数组，长度为1024，用来一次性读取多个字节
        byte[] b = new byte[1024];
        //len用来存储每次读取的字节数，以控制写入的字节数，防止结尾处填不满数组导致的多写
        int len;
        //循环读取文件，直到读完文件返回-1
        //直接把read方法的返回值给len，这个返回值就是这次读取的字节数
        while ((len=fis.read(b)) != -1){
            /*
            这一行表示用b数组从0索引开始，写入len个字节到fos中，这样就可以读多少写多少
            如果固定输出长度，那么如果在结尾处那一次没有刚好填满这个数组，
            上一次读取没有被覆盖的内容就会重复被输出，导致错误
            */
            fos.write(b,0,len);
        }
        //关闭流，由于输入流先打开，所以后关闭，输出流后打开，所以先关闭
        fos.close();
        fis.close();
    }
}
```

doro



Q2

读取name.txt 将表中的人名按照unicode自然排序 每个人名占据一行 在项目根目录下创建name_sorted.txt文件 将排序后的内容写入name_sorted.txt文件 要求去掉多余的首尾空格与空行

前言

- 先回答题目自带的问题吧：

为什么不建议手写换行符？

换行符在不同操作系统不同，比如windows是\r\n，老版本macos是\r，现在mac和linux是\n，用newLine方法可以自动生成他们，在不同系统都不会出错

然后是一些问题和反思

- 关于异常的处理

由于题目没有做出特别的要求，为了节省篇幅，我选择了直接抛出处理而非try-with-resources

- 关于导入类部分的省略

我用 `import java.io.*` 省略了所有io类的导入，不过也是为了节省篇幅

我用到的IO类有这些： `BufferedReader`； `BufferedWriter`； `FileInputStream`； `FileOutputStream`； `IOException`； `InputStreamReader`； `OutputStreamWriter`

- 关于排序的方法

我使用了**Treeset**集合自动排序，但确实会面临这两个问题：

1. set集合不允许重复，所以会帮我去重

2. TreeSet默认帮我以**UTF-16码元**进行排序，虽然它和UTF-8都是Unicode的转换格式，但是确实有不同

不过结合题目的情景，也没有什么问题

1. 本题没有重复的姓名，而且按理来说姓名不应重复
2. 就着题目字符排序内容的范围来说，**两个排序方式输出的顺序结果一样**，但如果来一些像emoji符号之类的可能就出问题了

- **关于IO类的写法**

其实FileReader这个字符流现在已经可以直接规定编码规则了吧，在jdk11以上就可以，所以感觉转换流在这上面的用途不大了，还要写很多嵌套(

- **我遇到的问题**

开始我以为trim会处理空行呢，并且我也没发现name.txt里的空行，于是我就没有加上空行跳过判断，结果读出来乱大了(

测试代码

```
package TASK1.Q2;

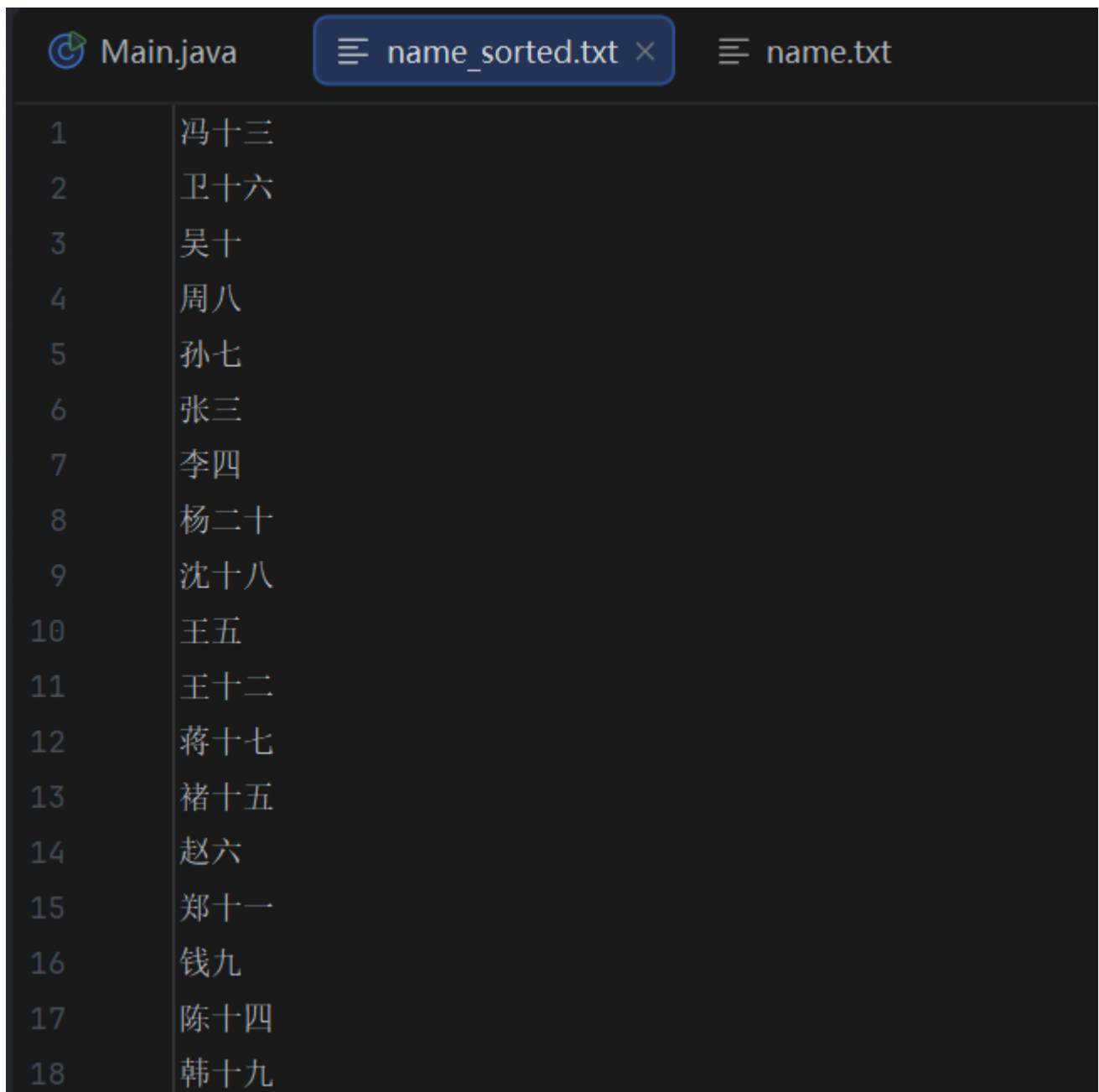
import java.io.*;
import java.nio.charset.StandardCharsets;
import java.util.TreeSet;

public class Main{
    public static void main(String[] args) throws IOException {
        /*
            (草稿)需要创建的流对象有:
            1.FileInputStream
            2.InputStreamReader
            3.BufferedReader
        */
        //写成这种嵌套形式，只用关一次流就可以自动关里面的底层流
        BufferedReader br = new BufferedReader(new InputStreamReader(new
        FileInputStream("./name.txt"), StandardCharsets.UTF_8));

        //接下来进行读取和排序，因为这里没有重复姓名，这里我用的TreeSet帮我自动排序
        //btw，虽然TreeSet默认用的UTF-16码元排序，源文件是UTF-8，但是顺序没问题
        String s;    //先创建一个字符串对象来接收
        TreeSet<String> set = new TreeSet<>();
        //开始循环读取数据
        while ((s=br.readLine())!=null){
            //文件里有个空行，我用if跳过它
            if (s.isBlank()){
                continue;
            }
            //先把前面那些空格去掉，然后add到集合里
            s=s.trim();
            set.add(s);
        }
    }
}
```

```
//接下来该输出了，先创建对象
BufferedWriter bw = new BufferedWriter(new OutputStreamWriter(new
FileOutputStream("./name_sorted.txt"), StandardCharsets.UTF_8));
//增强for遍历集合
for (String str : set){
    //我是windows系统，别的系统换行符不一样，这里用newLine方法换行
    bw.write(str);
    bw.newLine();
}
bw.close();
br.close();
}
```

name_sorted.txt



The image shows a code editor window with a dark theme. At the top, there are three tabs: 'Main.java', 'name_sorted.txt' (which is active and highlighted with a blue border), and 'name.txt'. The 'name_sorted.txt' tab contains 18 lines of text, each consisting of a number followed by a Chinese name. The lines are numbered 1 through 18 on the left side of the editor.

1	冯十三
2	卫十六
3	吴十
4	周八
5	孙七
6	张三
7	李四
8	杨二十
9	沈十八
10	王五
11	王十二
12	蒋十七
13	褚十五
14	赵六
15	郑十一
16	钱九
17	陈十四
18	韩十九

TASK 2

Q3

前言和笔记：

- **Serializable接口：**

这个接口没有任何抽象方法和内容，它的内容如下：

```
public interface Serializable {  
}
```

这是一个**标记型接口**，原文有这样一段解释：

Classes that do not implement this interface will not have any of their state serialized or deserialized. All subtypes of a serializable class are themselves serializable. **The serialization interface has no methods or fields and serves only to identify the semantics of being serializable.**

翻译：未实现此接口的类，其任何状态都不能被序列化或反序列化。可序列化类的所有子类本身也都是可序列化的。这个序列化接口没有方法和字段，仅用于标识“具备可序列化”这一语义。

所以，它的作用就是给类打上一个标识，说明这是一个可序列化的类，标给谁看呢？**其实是给JVM看**

JVM在执行序列化流输出的时候，会在这个.class文件里**检查这个类是否实现了Serializable接口**，如果有，那可以继续进行；但如果没有，就会抛出 `NotSerializableException`

- **序列化版本号：**

除了实现接口以外，我加上了一句

```
private static final long serialVersionUID = 1L;
```

这是给这个可序列化的类加上一个版本号，版本号是long类型的

如果不写版本号，JVM会根据我的类结构自动生成一个，但是JVM帮我生成的版本号在我每次**修改类的内容时都会改变**

版本号有什么用？在反序列化时，JVM会把文件里保存的版本号和类的版本号进行对比，只有相同才能进行反序列化，但如果不同，则会抛出 `InvalidClassException`

所以，为了方便地对类进行修改，可序列化的类我们最好都手动写上一个版本号（当然也可以让编辑器来生成一个固定的）

关于写版本号的语句：

1. 这一句习惯写在类的最上方，方便查看
2. 如果是1这种小数字，要加上L表示这是long类型更规范（我试了一下short竟然可以，不过还是规范一点）
3. 这个版本号是private权限的，static表示他属于类，还要加上final以让这个版本号无法被再次修改
4. btw，由于static字段，父类的版本号不会继承给子类，这是一个类独有的

- **transient 关键字**

一般来说，序列化一个类会把类里的成员变量全部写进文件里

可是如果需要控制部分成员变量**不被序列化**，不写入文件内，就要用这个transient关键字

写法如下：（举例说明）

如果我不想让手机号114514(?)被序列化，应该这么写：

```
private transient String phone;
```

这样phone这个成员变量就不会被序列化了！

Student类(实现了接口，加上了一个版本号)

```
package TASK2.Q3;

import java.io.Serializable;
//实现接口Serializable
public class Student implements Serializable {
    private static final long serialVersionUID = 1L; //版本号
    private Integer id;
    private String name;
    private Integer gender; //1表示男性 2表示女性
    private String phone;

    public Student(Integer id, String name, Integer gender, String phone) {
        this.id = id;
        this.name = name;
        this.gender = gender;
        this.phone = phone;
    }

    public Integer getId() {
        return id;
    }

    public void setId(Integer id) {
        this.id = id;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public Integer getGender() {
        return gender;
    }

    public void setGender(Integer gender) {
```

```

        this.gender = gender;
    }

    public String getPhone() {
        return phone;
    }

    public void setPhone(String phone) {
        this.phone = phone;
    }

    @Override
    public String toString() {
        return "Student{" +
            "id=" + id +
            ", name='" + name + '\'' +
            ", gender=" + gender +
            ", phone='" + phone + '\'' +
            '}';
    }
}

```

序列化

```

package TASK2.Q3;

import java.io.FileOutputStream;
import java.io.IOException;
import java.io.ObjectOutputStream;

public class Main0 {
    public static void main(String[] args) throws IOException {
        Student student = new Student(1, "doro", 2, "114514");
        ObjectOutputStream oos = new ObjectOutputStream(new
        FileOutputStream("student.dat"));
        oos.writeObject(student);
        oos.close();
    }
}

```

反序列化

```

package TASK2.Q3;

import java.io.FileInputStream;
import java.io.IOException;
import java.io.ObjectInputStream;

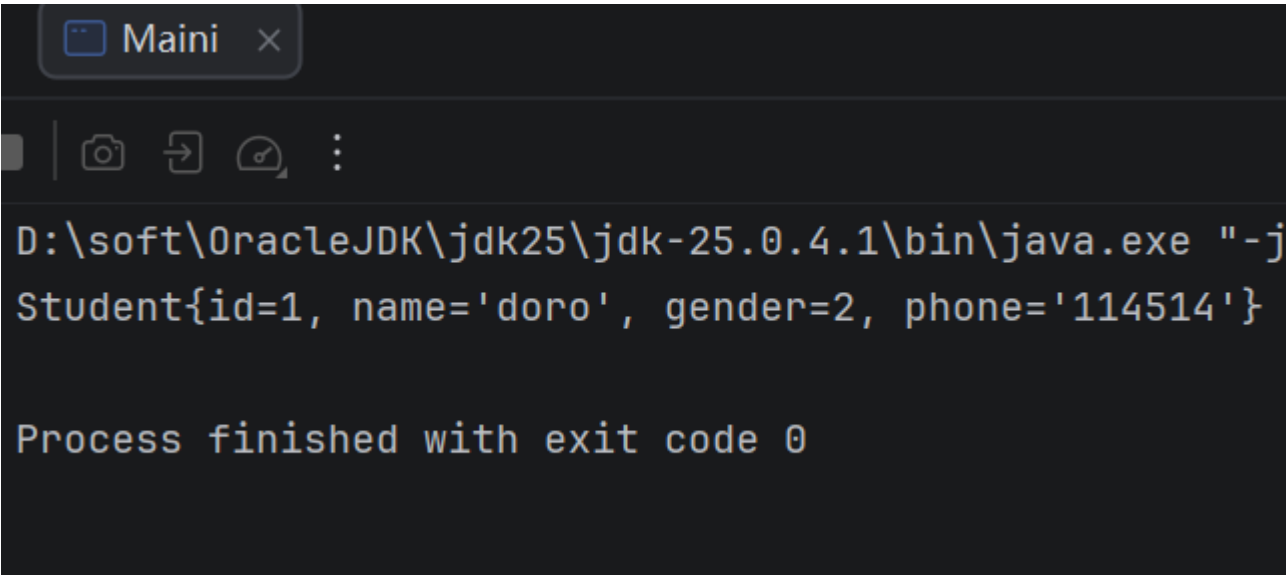
public class Main1 {

```

```
public static void main(String[] args) throws IOException, ClassNotFoundException {
    ObjectInputStream ois = new ObjectInputStream(new FileInputStream("student.dat"));
    Student student = (Student) ois.readObject();    //readObject最后返回Object类型的结果，
    强转为Student
    System.out.println(student);
    ois.close();
}
```

student.dat

反序列化后运行截图



TASK 3

前言

这个任务我当笔记用了ww

maven网上的教程真的很参差不齐，真的绕了半天才知道maven是干嘛的

IDEA关联maven那里，我开始都不知道自带maven，还以为那是自动识别这么智能呢(

不过那些生命周期命令现在还是不知道怎么使用，主要是不知道打包干啥，等以后能做项目了再来吧ww

• 什么是Maven?

Maven 是 Java 的项目构建与依赖管理工具，可以解决java开发的jar包依赖、版本等等过多（依赖管理）和手动编译、测试、打包过于麻烦（项目构建）等问题

• 什么是jar包?

jar全称Java Archive，是Java归档文件，后缀.jar，本质是一个压缩包

里面存放编译后的.class字节码文件、资源文件（配置、图片等）

• maven和jar包有关系吗？有什么关系？

有关系，maven就是用来**管理和生产**jar包的工具

如果没有maven，我们就要手动管理jar，此时如果某个jar需要另一个依赖文件jar，或下载了两个版本的同一jar，都会很麻烦

maven可以让我们直接在pom.xml里写依赖坐标自动地查找、下载、版本管理等

• 为什么要用maven?

1. 在写java项目的时候，如果没有maven，我们需要手动找jar包，手动将其添加到类路径，手动排查版本冲突、下载依赖，比较繁琐
2. 可以统一目录结构和生命周期，用一条命令完成编译、测试、打包，进行标准化
3. maven里pom.xml是唯一的构建配置，所有人环境一致，更适合协作
4. 有良好插件生态，编译、测试、打包、部署这类构建操作，都是由可插拔的插件完成的，更易于扩展

我的maven版本:3.10.0

```
Apache Maven 3.10.0 (c43a36b8d67be7e0805a411bc0898af1a51f5472)
Maven home: D:\apache-maven-3.10.0-bin\apache-maven-3.10.0
Java version: 25.0.4.1, vendor: Oracle Corporation, runtime: D:\soft\OracleJDK\jdk25\jdk-25.0.4.1
Default locale: zh_CN, platform encoding: UTF-8, time zone: GMT+08:00
OS name: "windows 11", version: "10.0", arch: "amd64", family: "windows"
```

• maven仓库

Maven 仓库是**存放 jar 包、插件等构建资源的位置**，主要有本地仓库、远程仓库，其中远程仓库还有中央仓库和私服

◦ 本地仓库

本地仓库是我电脑本地的文件夹，存放我下载的jar，maven查找依赖最先从本地仓库开始找

如何配置我自己的本地仓库？

在maven安装目录里有settings.xml,在大约50行的位置有这样一段注释

```
<!-- localRepository
| The path to the local repository maven will use to store artifacts.
|
| Default: ${user.home}/.m2/repository
<localRepository>/path/to/local/repo</localRepository>
```

这表示我的本地仓库目录默认在我的用户名文件夹内

把最下面一行给单独复制粘贴到注释外面，把中间改成自己要的地址就行比如我更改成：

```
<!-- localRepository
| The path to the local repository maven will use to store artifacts.
|
| Default: ${user.home}/.m2/repository
<localRepository>/path/to/local/repo</localRepository>
-->
<localRepository>D:\\maven\\repository</localRepository>
```

在IDEA设置里打开重写可以直接设定自己要的目录，也可以不打开override使用settings里的目录

◦ 远程仓库

在我们本地仓库没有对应的jar包时，maven会去远程仓库进行下载，再存入我的本地仓库（下次不用再去远程了）

远程仓库分为**中央仓库**和**私有仓库**

■ 中央仓库

其中中央仓库是maven官方自带的远程仓库，国内有阿里云镜像；而私有仓库是公司团队自己搭服务器来放项目内写的jar

在配置文件里，默认是官方的那个中央仓库，我可以在里面写自己要的镜像，把这一段直接放进mirrors里

```

<mirror>
  <id>maven-default-http-blocker</id>
  <mirrorOf>external:http:*</mirrorOf>
  <name>Pseudo repository to mirror external repositories initially using
HTTP.</name>
  <url>http://0.0.0.0/</url>
  <blocked>true</blocked>
</mirror>
<mirror>
  <id>aliyunmaven</id>
  <mirrorOf>*</mirrorOf>
  <name>阿里云公共仓库</name>
  <url>https://maven.aliyun.com/repository/public</url>
</mirror>

```

■ 私服

私服是搭建在私有服务器上的远程仓库，一般是公司来搭建，它可以用来存放公司内部写的jar，方便团队协作开发和应对一些jar不能放在公网上的情况，比较常用的有Nexus、Artifactory等等

• Maven项目

在IDEA的setting里可以找到maven的设置，IDEA里自带一个maven，我改成了自己的maven目录

◦ maven项目的参数

maven项目的参数主要是**坐标**，此外还有dependencies依赖管理，properties属性等

■ 坐标

maven的坐标有三个点：**groupId(组织名/包名)**、**artifactId(模块名)**、**version(版本号)**

其中：

groupId表示项目所属团队、组织，一般是照着公司的域名反向命名，是不会轻易改动的

artifactId是当前模块名称，在同一个groupId内不可以重名

而version是版本号，一般有两种类型，release正式发行版，snapshot快照版

在父子工程中，子模块可以省略这个版本号

此外，还有 `<packaging>` 配套，里面可以设定成打包为jar包或者war包。如果里面写成pom，表示这是父工程，这样它就不会进行任何打包操作

■ dependencies

`<dependencies>` 是**所有依赖的外层容器**，里面放多个 `<dependency>`，**一个 dependency 对应一个 jar 包**

其中，可以在 `<scope>` 里写它的**依赖范围**，主要有compile、test、provided、runtime、system等

1. compile默认

编译、测试、运行全部会生效，也会打包进jar

2. test测试

仅在测试阶段生效，不会打包进jar

3. provided

编译、测试阶段生效，不会打包进jar

4. runtime

运行时生效，会打包进jar

5. system

这个范围要额外写一个systempath，表示使用本地硬盘上的jar，不常用

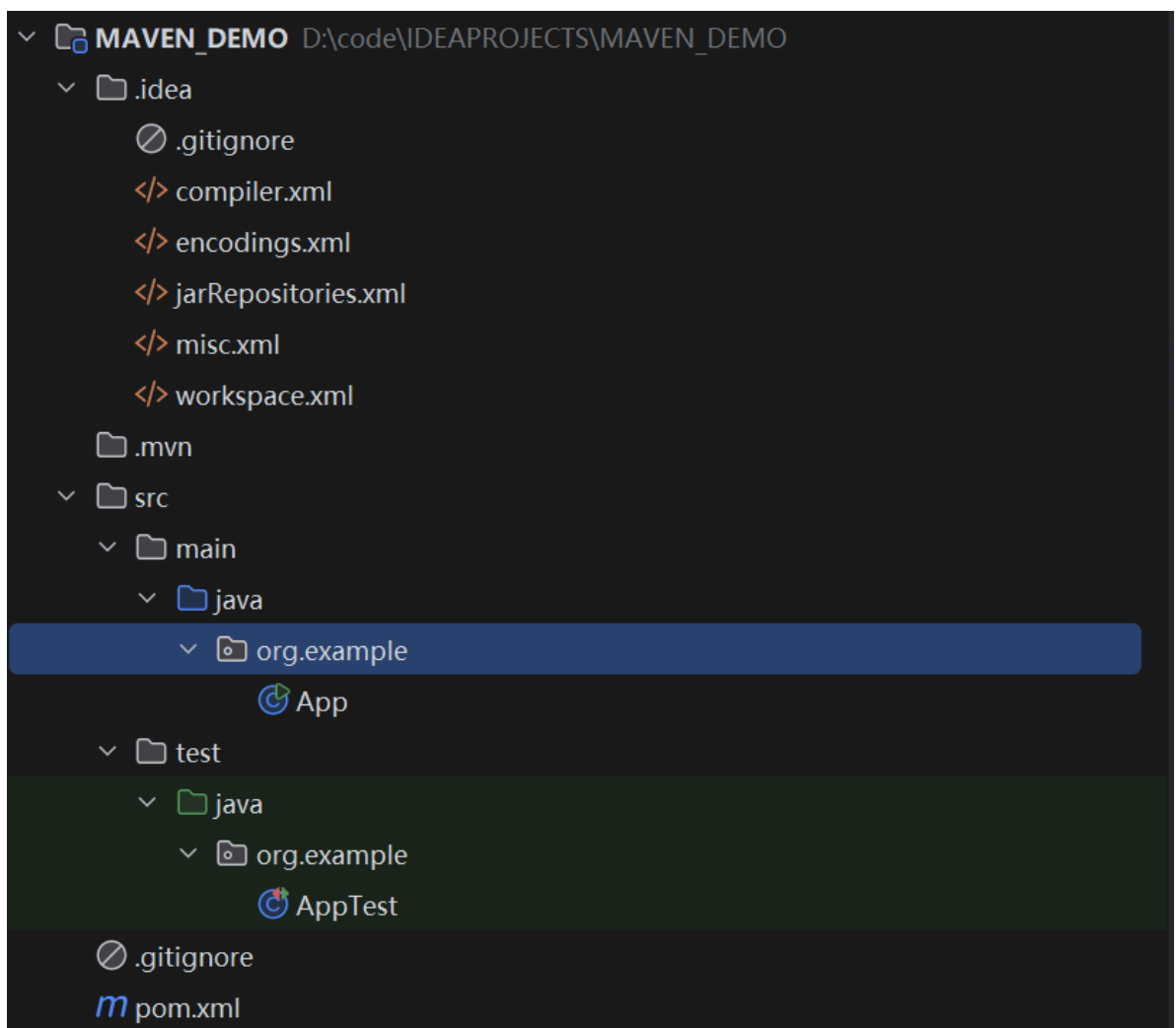
此外，还可以使用**exclusion**排除依赖，解决需要同一依赖不同版本的冲突

■ properties

properties可以定义一个**全局变量**，使用 `${属性名称}` 引用

在多处使用同一依赖同一版本时，如果需要升级版本，就只需要改这一处而非在各个地方一一改动

○ maven项目结构



这是从IDEA里直接看到的我随便创的一个maven项目的结构

其中我没有创建resources文件夹

1. pom.xml文件

在项目根目录下，是Maven项目配置文件

2. src目录

是Maven项目的主要**源代码和资源**存放位置，有两个子目录：main和test

3. src.main

包含项目**主代码、资源文件**

其中，src.main.java里面放**项目Java源代码文件和各种类**，在编译阶段，maven会自动编译目录里的所有java文件

如果创建了**资源文件夹**，会放在main目录下 `src.main.resources` 用来存放**配置文件**等

4. src.test

用于存放**测试代码和测试配置文件**，其中：

src.test.java存放src.main.java的类对应的**测试代码**

src.test.resources存放测试相关的**测试配置文件**等

此外，在项目根目录里还会自动生成target目录，**存放编译后的 class、jar 包、临时文件**

所以总结一下就是：

全部代码、配置文件放在src里

与主代码有关的放在main里

与测试相关的放在test里

源代码在java，配置文件在resources，编译文件放在target，**项目配置文件是pom.xml**

◦ pom.xml

■ 什么是pom.xml?

pom是Project Object Model项目对象模型，pom.xml是maven**项目的核心描述文件**，放在**项目根目录下**

■ 它有什么用?

maven会读取pom.xml文件，获取项目的配置参数（比如坐标）、需要的jar包、如何进行编译打包等

总的来说，它主要就是以下几个功能：

1. **定义项目的坐标**，让项目有唯一身份
2. **管理第三方jar包**，自动下载、版本管理，依赖处理
3. **配置项目信息**，比如JDK版本，我这里改成了我的JDK25
4. **管理打包**，比如定义打包成jar还是war

■ 基本配置（详细见上面maven项目的参数）

1. **项目坐标**
2. packaging项目类型
3. properties属性（主要写jdk版本、编码格式等）
4. dependencies依赖管理

■ 如何用它管理jar包版本、项目类型?

其实前面已经有提到了

关于jar包版本：

在导入jar包之后，在version坐标处写它的版本就行

可以先在properties里写同一版本，然后再version处引用（引用方法见上）

关于项目类型管理：

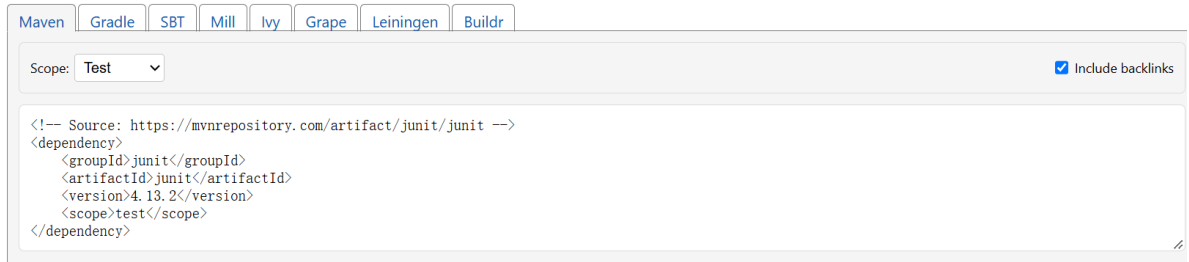
在 <packaging> 标签里改动即可，可以改为jar/war/pom，默认为jar

◦ 那么如何导入我要的jar包呢？

直接找一个查依赖坐标的网站，我用<https://mvnrepository.com/>

然后搜索我要的jar包，以junit为例

再找到最新的稳定版本，点进去，它会给你一个dependency片段



然后把它复制粘贴进pom.xml的 <dependencies> 里就行，然后IDEA右边面板点刷新

在这里只需要代入直接依赖即可，间接的依赖maven会自动帮我下载

如果出现了依赖冲突，会根据首先最短路径优先，其次先声明的优先的顺序来确定优先使用哪个版本，可以手动处理

• maven的常用命令

主要命令的格式：mvn 命令内容

maven的生命周期分clean、default、site三个

其中default（主要）含validate→compile→testcompile→test→package→install→deploy

site主要作用为读取 pom 信息、测试报告并生成项目文档网站

1. mvn clean

起删除作用，删除target文件夹里的编译文件

2. mvn compile

编译src.main.java的代码，把产生的class文件存入target

3. mvn test

先执行compile，再编译src.test.java里的测试代码，并运行junit测试

4. mvn package

主要作用为把项目打包，会执行compile、test、打包的生命周期

5. mvn install

在package流程的基础上，再把打好的包安装到本地仓库

6. mvn deploy

在install流程的基础上，把包上传到远程私服

7. 其他命令

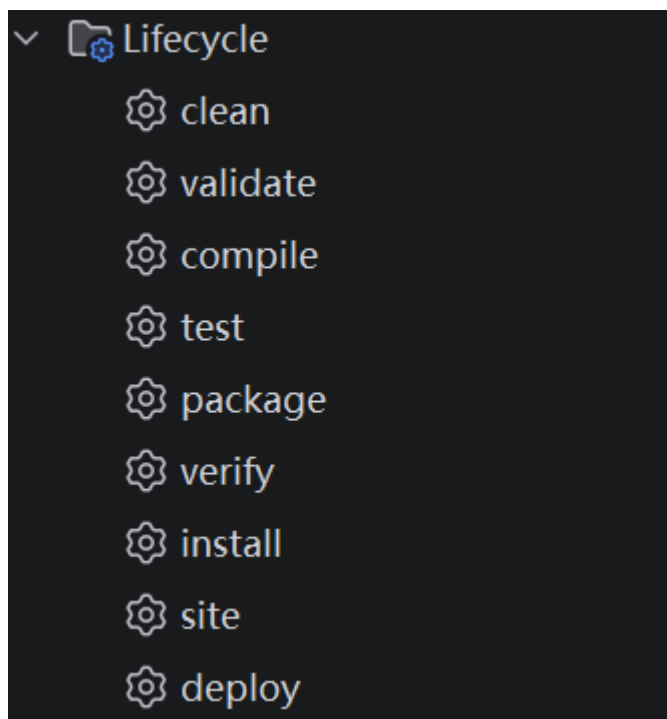
1. 组合命令

比如`mvn clean package`，先清理再打包，比较常用

2. `mvn dependency:tree`

打印依赖树，便于排查依赖冲突

IDEA里右边的面板里有现成的maven指令，只需要双击就可以了（面板左上角还可以刷新）



TASK 4

[准备](#)

[思路梳理](#)

[代码部分跳转](#)

[测试截图](#)

准备

有太多新知识点了，这里稍稍总结一下重点内容便于后续操作

• HTTP

HTTP，是应用层协议，用于客户端与服务端之间的**数据传输**，是每次请求都独立的无状态协议（服务器默认不保存请求间的状态）

如果要维持状态，需要借助**Cookie**等工具

○ HTTP请求报文

结构：

```
请求行
请求头
空行（用于分割）
请求体
```

请求行格式：方法 路径 HTTP版本→ `POST /login HTTP/1.1`

常用的请求方法：GET/POST

GET方法主要用于获取数据，特点是把**请求参数放在 URL 的查询字符串中**，会在日志，浏览记录里出现；GET方法是安全方法，不会修改服务器状态

而POST方法用来提交/创建，会修改服务器状态，是非安全方法，特点是把**参数放在请求体中**

幂等性：指同一个请求，执行一次和执行多次后**对服务器状态的最终效果一样**

这里GET方法是有幂等性的，而POST没有。因为GET不论请求多次都是查询，而POST就不是，重复请求可能造成问题

此外，还有PUT（用于更新或替换资源）、PATCH（用于部分更新资源）、DELETE（删除资源）等等等等

请求头：字段名: 字段值→ `Host: api.example.com`、`Content-Type: application/json`

最最常用的请求头：

`Host`：目标主机

`Content-Type`：请求体类型

`Content-Length`：请求体长度

`Authorization`：认证信息

请求体：可以是JSON、表单、文件等等，一般用Content-Length规定读取长度→ `{"username": "admin", "password": "123456"}`

GET方法通常没有请求体，而POST通常有body

■ HTTP响应报文

结构：

```
状态行
响应头
空行
响应体
```

状态行：HTTP版本 状态码 原因短语→ `HTTP/1.1 200 OK`

状态码主要按开头数字区分：

2开头为正常，成功：`200 成功`、`201 已创建`、`204 成功，无内容`

3开头为重定向：301 永久重定向、302 临时重定向

4开头为客户端问题：400 请求错误、401 未认证、403 已认证但是无权限、404 资源不存在、405 方法不允许

5开头为服务端问题：500 服务器内部错误、502 网关错误、503 服务不可用

响应头：与请求头格式相同→Content-Type: application/json

最最常用的响应头：

Content-Type：响应体类型

Content-Length：响应体长度

Cache-Control：缓存策略

Location：重定向地址

响应体：是服务器返回给客户端的数据，可以是JSON、HTML等→{"status": "success", "token": "xyz"}

• JSON

JSON是一种**轻量级的数据交换格式**，用文本表示结构化的数据，利于人和机器双方的读写

◦ JSON基本结构

主要有两种

- **对象：**用 {} 表示，里面是键值对
- **数组：**用 [] 表示，里面是一组值

注意点：键必须是双引号字符串、不能有注释、不能有尾随逗号

值的类型：字符串、数字、布尔、null、对象、数组

在HTTP里的体现

请求头响应头里写Content-Type: application/json

请求体响应体可以用JSON

◦ fastjson

这是一个常用的**java的JSON解析库**，实现JSON字符串和java对象的互相转换

对象到字符串：String json = JSON.toJSONString(obj);

字符串到对象：User user = JSON.parseObject(json, User.class);

• HTTP有关的java类和方法

以下用在服务端

◦ HttpServer类

1. .create(InetSocketAddress addr, int backlog)

创建HTTP服务器实例，绑定IP + 端口，开启TCP监听

第一个参数写**绑定的地址和端口**，比如写 new InetSocketAddress(8000) 就是监听本机所有网卡8000端口

第二个参数是**TCP连接等待队列长度**，0是jdk默认值

2. `.createContext(String path, HttpHandler handler)`

注册路由，指定路径交给对应的处理器处理

第一个参数是请求路径，只有路径匹配才能交给handler

第二个参数是一个处理器，里面一般填一个实现了**HttpHandler接口**的类的对象

3. `.start()`

启动服务器，开始监听端口，等待客户端连接

○ **HttpHandler接口**

只有一个抽象方法：`void handle(HttpExchange exchange)`

当有HTTP请求到达匹配的路径时，JDK 自动调用这个方法，实现类重写这个方法为请求之后的处理逻辑

参数为本次HTTP请求和响应的封装对象，拿取**本次请求的所有信息**

○ **HttpExchange类**

1. `String getRequestMethod()`

返回字符串，用来判断请求方式，比如GET或者POST

2. `InputStream getRequestBody()`

返回一个输入字节流，**读取客户端发来的 POST 请求体**，记得关流（可以写成try-with-resources自动关）

请求体得是字节，如果用JSON字符串，**要先转成字节数组才行**

3. `void sendResponseHeaders(int rCode, long responseLength)`

用来发送HTTP状态码，给响应头

第一个参数是**HTTP状态码**，比如404

第二个参数是**响应体的字节长度**，填响应JSON对应的**字节数组长度**，如果填-1那就是没有响应体，0就是长度未知分块传输

4. `OutputStream getResponseBody()`

用来获取响应体输出流，返回一个输出字节流，还是要记得关流

与getRequestBody类似，这里也要往里面写字节

5. `void close()`

结束本次 HTTP 会话，一次请求处理完成后必须调用

以下用在客户端

○ **URL类**

1. 构造方法 `URL(String spec)`

这个参数是填完整的url字符串：**协议:// 地址: 端口 / 路径**

○ **HttpURLConnection类**

由url.openConnection()得到

1. `url.openConnection()`

打开HTTP连接，返回连接对象

```
URLConnection connection = (URLConnection) url.openConnection();
```

这里是把返回的连接对象强转为URLConnection

2. `setRequestMethod(String method)`

用来**设定HTTP请求方式**

参数直接写方法对应的字符串，比如"POST"

3. `setRequestProperty(String key, String value)`

用来**设置HTTP请求头**

第一个参数是请求头的key，第二个是请求头的value

4. `setDoOutput(boolean doOutput)`

用来设置是否开启输出，**是否允许写POST请求体**，这个默认false

`setDoInput`是默认true

5. `OutputStream getOutputStream()`

返回一个输出字节流，用来把POST的请求体JSON字节写入，通过网络发给服务端

6. `int getResponseCode()`

返回int，获取HTTP响应状态码

7. `InputStream getInputStream()`

返回一个输入字节流，在成功响应（2开头）时，读取服务端返回的响应体

8. `InputStream getErrorStream()`

出错时（4/5开头）读取错误响应内容

9. `void disconnect()`

关闭 HTTP 连接，释放底层 TCP 资源

梳理一下思路

[跳转到下面的代码](#)

这里我把要做什么按顺序写一下，并把引申出来的一些需要斟酌的问题拿出来

- 创建项目，pom.xml导入fastjson依赖
- 编写类的内容，四个类顺序优先级请求>响应，被依赖的先，依赖别人的后

那么就QueryRequest→QueryResponse→Server→Client

- **QueryRequest**

用来封装客户端发过来的查询参数：快递单号、手机号

- **需要什么成员变量？** trackingNumber单号和phone手机号
- **需要什么方法？**
 1. 构造方法（应该不需要有参）
 2. get和set

- **QueryResponse**

用来封装服务端返回给客户端的查询参数

- **成员变量?** pickCode取件码和msg提示文本
- **方法?**
 1. 构造方法
 2. getset

- **Server**

这个类主要由一个初始化方法和一个QueryHandler内部类组成

- **存数据用什么集合?**

HashMap肯定是，首先这是键值对存储数据，再者hashmap查找快
- 题目给的key格式：快递单号_手机号；value：取件码
- **TODO1**：直接照着那一行把数据put进去
- **处理器QueryHandler**
 1. 非POST请求返回405

直接if比较，不过"**POST**"要写前面不然可能空指针异常
 2. 如果是POST那就开始获取请求体
 3. 把它这个JSON反序列化为对象QueryRequest
 4. **TODO2**：校验单号和手机号不能为空

直接加一个判断，条件就是四个，分别是单号和手机号不为空格和不为null
如果空了直接throw，用IllegalArgumentException
 5. **TODO3**：拼key查询取件码

从map里把用get把数据拿出来拼好就行
 6. **TODO4**用set方法就行，没有就写null
 7. **TODO5**：把响应对象序列化为JSON并发送

用jsonRes变量接收JSON.toJsonString方法
然后先把这个字符串转成字节流才能输出，用getBytes
然后发送200状态码，把响应JSON写入响应流，用trywithresources自动关流
 8. **TODO6**同理

- **Client(简略写了，很多地方类似)**

这个类由main和读取响应体的方法构成

1. **TODO1**判断一下就好
2. **TODO2**: 用fastjson构造json格式的请求体

先创建QueryRequest的对象，再set方法写把手机号和取件码写进去最后转json的请求体
3. **TODO3**：先创建URLConnection连接对象，再设定请求方法，然后写请求头设定是json格式
接着写入请求体并读取响应
4. 根据能否拿到取件码，打印查询的结果（比如todo4，对了就把json转回来，然后打印就行

代码

代码的注释由于篇幅太长就简写了，主要内容还是在上面的梳理www

[思路梳理](#)

我写完才发现有个变量pick_code写成pickCode了！这就不改了叭

QueryRequest

```
public class QueryRequest {
    private String trackingNumber;
    private String phone;
    //构造
    public QueryRequest(){
    }
    //getset
    public String getTrackingNumber() {
        return trackingNumber;
    }
    public void setTrackingNumber(String trackingNumber) {
        this.trackingNumber = trackingNumber;
    }
    public String getPhone() {
        return phone;
    }
    public void setPhone(String phone) {
        this.phone = phone;
    }
}
```

QueryResponse

```
public class QueryResponse {
    private String pickCode;
    private String msg;
    //构造
    public QueryResponse(){}

    public QueryResponse(String pickCode, String msg){
        this.pickCode = pickCode;
        this.msg = msg;
    }
    //getset
    public String getPickCode() {
        return pickCode;
    }
    public void setPickCode(String pickCode) {
        this.pickCode = pickCode;
    }
    public String getMsg() {
        return msg;
    }
}
```

```

    }
    public void setMsg(String msg) {
        this.msg = msg;
    }
}

```

Server

```

import com.alibaba.fastjson2.JSON;
import com.sun.net.httpserver.HttpExchange;
import com.sun.net.httpserver.HttpHandler;
import com.sun.net.httpserver.HttpServer;

import java.io.IOException;
import java.io.OutputStream;
import java.net.InetSocketAddress;
import java.nio.charset.StandardCharsets;
import java.util.HashMap;
import java.util.Map;

public class Server {
    private static final int PORT = 8000;
    // 模拟快递柜数据: key = 快递单号_手机号, value = 取件码
    private static final Map<String, String> expressMap = new HashMap<>();

    public static void main(String[] args) throws IOException {
        // 初始化一些测试数据
        initializeExpressData();
        // 创建HTTP服务器, 监听指定端口
        HttpServer server = HttpServer.create(new InetSocketAddress(PORT), 0);
        // 设置路由和处理程序
        server.createContext("/query", new QueryHandler());
        // 启动服务器
        server.start();
        System.out.println("Server started on port " + PORT);
    }

    private static void initializeExpressData() {
        // 添加一些测试数据
        // 键的构成是 快递单号_手机号
        expressMap.put("SF123456789_13005433678", "1234");
        // TODO 1: 参照测试数据表, 把其余3条数据也加进去
        expressMap.put("JD987654321_19805433168", "5678");
        expressMap.put("YT456789123_13905479698", "9012");
        expressMap.put("ZT789123456_18505433664", "3456");
    }

    static class QueryHandler implements HttpHandler {
        @Override
        public void handle(HttpExchange exchange) throws IOException {
            // 非POST请求返回405 Method Not Allowed
            if (!"POST".equals(exchange.getRequestMethod())) {

```

```

        exchange.sendResponseHeaders(405, -1);
        exchange.close();
        return;
    }

    try {
        // 读取请求体（请求体是字节流，按UTF-8转成字符串）
        String body = new String(exchange.getRequestBody().readAllBytes(),
StandardCharsets.UTF_8);
        // 解析JSON请求，取出单号和手机号
        QueryRequest request = JSON.parseObject(body, QueryRequest.class);
        String trackingNumber = request.getTrackingNumber();
        String phone = request.getPhone();

        // TODO 2: 校验单号和手机号不能为空，不合法就抛异常走400分支
        //四个条件，isBlank用来看空格，还要防null
        if (trackingNumber == null || phone == null || trackingNumber.isBlank() ||
phone.isBlank()) {
            throw new IllegalArgumentException("单号和手机号不能为空");
        }
        // TODO 3: 用"单号_手机号"拼成key，在expressMap中查询取件码
        String pickCode = expressMap.get(trackingNumber + "_" + phone);
        // TODO 4: 根据查询结果构造QueryResponse
        //先创建一个变量，后面再new不用手动重新赋值
        QueryResponse response;
        if (pickCode != null){
            response = new QueryResponse();
            response.setPickCode(pickCode);
            response.setMsg("success");
        }else{
            response = new QueryResponse();
            response.setPickCode(null);
            response.setMsg("未找到取件码");
        }
        // TODO 5: 把响应对象序列化成JSON并发送响应（状态码200）
        String jsonRes = JSON.toJSONString(response);
        //先转成字节数组
        byte[] resBytes = jsonRes.getBytes(StandardCharsets.UTF_8);
        exchange.sendResponseHeaders(200, resBytes.length);
        //try-with-resources自动关流
        try (OutputStream os = exchange.getResponseBody()) {
            os.write(resBytes);
        }
        exchange.close();
    } catch (Exception e) {
        // 处理异常，返回400状态码(Bad Request)
        // TODO 6: 返回状态码400和错误JSON {"pick_code":null,"msg":"请求格式错误"}
        //这里和TODO 5很类似
        String errorjson = JSON.toJSONString(new QueryResponse(null, "请求格式错误"));
        byte[] errorBytes = errorjson.getBytes(StandardCharsets.UTF_8);
        exchange.sendResponseHeaders(400, errorBytes.length);
        try (OutputStream os = exchange.getResponseBody()) {
            os.write(errorBytes);
        }
    }
}

```

```

    }
    exchange.close();
}
}
}
}
}

```

Client

```

import com.alibaba.fastjson2.JSON;

import java.io.BufferedReader;
import java.io.IOException;
import java.io.InputStream;
import java.io.InputStreamReader;
import java.io.OutputStream;
import java.net.HttpURLConnection;
import java.net.URL;
import java.nio.charset.StandardCharsets;
import java.util.Scanner;

public class Client {
    private static final String SERVER_URL = "http://localhost:8000/query";

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        while (true) {
            try {
                // 读取用户输入的快递单号
                System.out.print("请输入快递单号（输入exit退出）：");
                String trackingNumber = scanner.nextLine().trim();
                // TODO 1: 输入exit时退出循环（
                // 直接对比一下字符串就好
                if ("exit".equalsIgnoreCase(trackingNumber)) {
                    break;
                }

                // 读取用户输入的手机号
                System.out.print("请输入手机号：");
                String phone = scanner.nextLine().trim();

                // TODO 2: 用fastjson构造json格式的请求体，样例为
                // {"trackingNumber":"SF123456789","phone":"13005433678"}
                QueryRequest req = new QueryRequest();
                // 先把数据set好
                req.setTrackingNumber(trackingNumber);
                req.setPhone(phone);
                // 一起转了
                String jsonBody = JSON.toJSONString(req);

                // TODO 3: 创建HttpURLConnection，把上面的json写入请求体并发送HTTP POST请求
                // （注意：连接对象要命名为connection，下面判断状态码要用）
                URL url = new URL(SERVER_URL);
            } catch (Exception e) {
                e.printStackTrace();
            }
        }
    }
}

```

```

//创建连接对象，名字是connection
URLConnection connection = (URLConnection) url.openConnection();
//设定方法和请求头
connection.setRequestMethod("POST");
connection.setRequestProperty("Content-Type", "application/json");
//改为true，允许写请求体
connection.setDoOutput(true);
//开流
try (OutputStream os = connection.getOutputStream()) {
    os.write(jsonBody.getBytes(StandardCharsets.UTF_8));
    os.flush();
}
// 读取响应，状态码是200才是响应成功
if (connection.getResponseCode() == 200) {
    // TODO 4: 读取响应体并解析json，拿到取件码就打印取件码，
    // 拿不到就打印msg的内容。响应样例：
    // {"pick_code":"1234","msg":"success"}
    // {"pick_code":null,"msg":"手机号不正确"}
    String responseBody = readBody(connection.getInputStream());
    QueryResponse response = JSON.parseObject(responseBody,
QueryResponse.class);
    //判断一下
    if (response.getPickCode() != null) {
        System.out.println("取件码: " + response.getPickCode());
    }else{
        System.out.println("查询失败, " + response.getMsg());
    }
}else{
    System.out.println("查询失败, 状态码: " + connection.getResponseCode());
}
connection.disconnect();
} catch (IOException e) {
    // 请求发生错误（比如Server没启动），打印错误信息后继续循环
    System.out.println("请求发生错误: " + e.getMessage());
}
}
scanner.close();
System.out.println("客户端已退出");
}

/** 读取响应体：把InputStream按UTF-8读成一个字符串 */
private static String readBody(InputStream is) throws IOException {
    StringBuilder sb = new StringBuilder();
    try (BufferedReader reader = new BufferedReader(new InputStreamReader(is,
StandardCharsets.UTF_8))) {
        String line;
        while ((line = reader.readLine()) != null) {
            sb.append(line);
        }
    }
    return sb.toString();
}
}

```

测试图片

```
请输入快递单号（输入exit退出）： YT456789123|
请输入手机号： 13905479698
WARNING: A terminally deprecated method
WARNING: sun.misc.Unsafe::arrayBaseOffs
WARNING: Please consider reporting this
WARNING: sun.misc.Unsafe::arrayBaseOffs
取件码： 9012
请输入快递单号（输入exit退出）：
```

```
请输入快递单号（输入exit退出）： 114514
请输入手机号： 1919810
查询失败，未找到取件码
```

```
请输入快递单号（输入exit退出）：
请输入手机号： 114514
查询失败，状态码： 400
```

```
请输入快递单号（输入exit退出）：
请输入手机号：
查询失败，状态码： 400
```